

Rajarata University of Sri Lanka
Department of Computing



ICT1402 PRINCIPLES OF PROGRAM DESIGN & PROGRAMMING

Lecture 02

Computer Program Design

K.A.S.H. Kulathilake

Ph.D., M. Phil., MCS, B.Sc. (Hons) IT, SEDA(UK)

Objectives

- At the end of this lecture students should be able to;
 - Define the phases of the program development life cycle.
 - Prepare task lists for solving given problems.
 - Define structured program theorem.
 - Apply structured program theorem when preparing a task list for a given problem.

Think on it?

What is
Coding?



Start the Work with some Activities?

- Activity 1:
 - Preparing a Tea



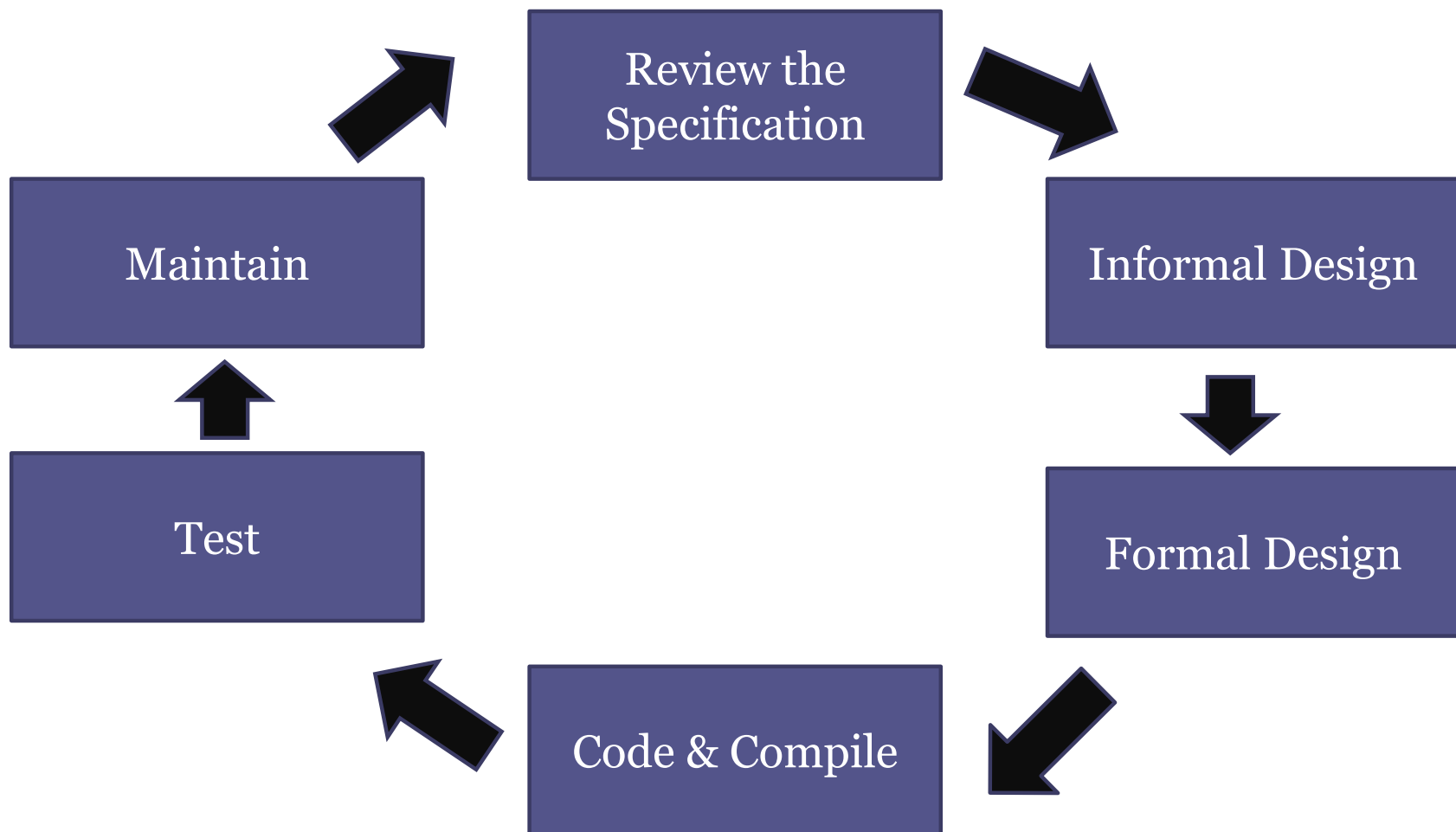
Tea bags
Boiled water
Sugar
Milk powder
Empty cup
Tablespoon

Start the Work with some Activities?

- Let's discuss



Program Development Cycle



Review Specification

- In this phase, we define the problem statement and we decide the boundaries of the problem.
- When you review the specification, ask yourself;
 - Do you understand what is required?
 - Do you have sufficient instruction to carry it out?
 - Do you have the necessary input available to generate the desired output?

Informal Design

- Addresses **WHAT need to be done.**
- **Do experiment on variety of possible solutions** and then decide on a general approach, before you formalize those ideas in a design.
- The informal design step **lets you make changes in your plan** before you invest too much time in one approach.
- There are two steps in the informal design that need to be completed frequently in a repeated cycle.
 - List the major tasks
 - List the subtasks

Informal Design (Cont...)

- List the major tasks
 - Identify the high-level tasks that needed to reach the expected output.
- List the sub tasks
 - Now, go back to each major tasks, one at a time, and list the subtasks for each major task.
- Once you have completed the major tasks and sub tasks list, you should figuratively **step back a bit, re-read the specifications, and make sure you have met all the requirements in the specifications.** If not go for modifications.
- Remember, you are not committed to the first attempt and frequently the first attempt will not be the best solution.

Informal Design (Cont...)

- Play with your lists a bit until you are satisfied that you have included everything in a logical approach.
- **Keep in mind that there is no single, right answer.**



Formal Design

- It takes the lists and puts them into a recognized format that others could easily read and use.
- Formal design **addresses HOW it will be done.**
- Formal design **is a communication tool for program designers and coders.**
- Eventually the formal design will **become a part of the program's documentation package** that will stay with the program as long as it is in use.
- There are many possible program design tools available and this course will concentrate on two of them namely; Flowcharts and Pseudocodes.

Formal Design (Cont...)

- Desk check the design
 - Desk checking is the way to check the logic of the design.
 - You will work through the design, step by step, implementing each step as the computer would, reading the test data, doing any calculations, and writing output.
 - This will tell you if there are any logic errors before you invest the time in coding. (assuming you have created good test data.)
 - If you have errors, go back to the task list, make the necessary changes, implement those changes in formal design, and try the desk checking again.

Formal Design (Cont...)



The informal design and formal design are really a cycle that you perform until your desk checking tells you the logic is perfect.

Code & Compile

- Coding the program is simply translating each design statement into the proper syntax for the desired programming language.
- In this phase, we construct actual program.
- That means we write the program to solve the given problem using programming languages like C, C++, Java etc.

Test

- During this phase, we check whether the code written in previous step is solving the specified problem or not.
- **That means we test the program whether it is solving the problem for various input data values or not.**
- We also test that whether it is providing the desired output or not.

Test (Cont...)



All those other Programmers who did not plan ahead frequently spend most of their program development time trying to get the "BUGS" out of their programs.

Maintenance

- During this phase, the program is actively used by the users.
- If **any enhancements found in this phase**, all the phases are to be repeated again to make the enhancements.
- That means in this phase, the solution (program) is used by the end user.
- If the user encounters any problem or wants any enhancement, then we need to repeat all the phases from the starting, so that the encountered problem is solved or enhancement is added.

Checklist for Preparing a Task List

- What output do you need to generate?
- What input do you have/ need?
- What form will it be in?
- Will the program have a list of tasks that needs to be done repeatedly (LOOP)?
- If it has repeated tasks;
 - What will be done within it?
 - What controls the loop?
 - What needs to be done before the loop?
 - What needs to be done after the loop?
- What are the other major tasks of the program?

Answers for these questions will give you the major task list. Then you can find the sub-tasks and sub-sub tasks.



Let's Prepare a Cup of Milk Tea

- What output do you need to generate? A cup of tea
- What input do you have/ need? Sugar, Milk, Tea, Water , Cup
- What form will it be in? Dry sugar, Tea bag, Milk powder, Boiled water , Empty cup
- Will the program have a list of tasks that needs to be done repeatedly (LOOP)? No
- What are the other major tasks of the program?
 - Take the empty cup
 - Add milk powder in to the cup
 - Add sugar in to the cup
 - Add boiled water into the cup
 - Stir 1 minute time
 - Add tea bag in to the cup and keep 1 minute time
 - Remove the tea bag from the cup
 - Stir 1 minute time to mix tea with milk

Let's Prepare a Cup of Milk Tea with a Condition

- You know most people would like to have a tea with sugar.
- But some people prefer tea without sugar due to health condition.



Easy..
Let's talk about
how to handling
selections in
problem solving....



How do we
address this
real world
condition in
tea making
problem?

Let's Prepare a Cup of Milk Tea with a Condition(Cont...)

- In most of the real-world problem solving we have to **make appropriate decisions to reach our goal.**
- It occurs in a conditional situation.
- In here, the action we perform will depend on the decision or selection we made.
- In our example; you have to decide whether you add sugar to the cup or not. It's governed by the personal preference due to the health condition of you. Simply you have two alternatives upon this "PERSONAL PREFERENCE" condition which are "YES" or "NO".
- If "PERSONAL PREFERENCE" is YES, you add sugar otherwise you do not add sugar.

Let's Prepare a Cup of Milk Tea with a Condition(Cont...)

- Selection constructs using two building blocks;
 - Condition that govern by the Boolean logic.
 - “PERSONAL PREFERENCE”
 - Possible actions under two alternative cases if possible.
 - If “PERSONAL PREFERENCE” is YES
 - You add sugar.
 - Else
 - You do not add sugar.

Let's Prepare a Cup of Milk Tea with a Condition(Cont...)



- Your sequential flow of tasks are branched into two paths due to a conditional situation.
- It selects one path at a time depending on the execution of the condition.
- We call the control of the sequential execution of the task list is changed due to the selection done at the conditional point.
- If condition is executed as true, the tasks in the true path will execute sequentially.
- Otherwise it executes the tasks in the false path.
- At the end of executing the tasks in the selected branch, the control of executing the task list connects with the remaining tasks in the problem.

Let's Prepare a Cup of Milk Tea with a Condition(Cont...)

- Relational Operations for Managing Conditions
 - $<$ → Less than
 - $<=$ → Less than or equal
 - $>$ → Greater than
 - $>=$ → Greater than or equal
 - $==$ → Equals
 - $!=$ → Not equal

Let's Prepare a Cup of Milk Tea with a Condition(Cont...)

- What output do you need to generate? A cup of tea
- What input do you have/ need? Sugar, Milk, Tea, Water , Cup
- What form will it be in? Dry sugar, Tea bag, Milk powder, Boiled water , Empty cup
- Will the program have a list of tasks that needs to be done repeatedly (LOOP)? No
- What are the other major tasks of the program?
 - Take the empty cup
 - Add milk powder into the cup
 - User preference for adding sugar is
 - Yes: Add sugar into the cup
 - No: Do not add sugar into the cup
 - Add boiled water into the cup
 - Stir 1 minute time
 - Add tea bag into the cup and keep 1 minute time
 - Remove the tea bag from the cup
 - Stir 1 minute time to mix tea with milk

Want More Control?

- Take the empty cup
- Add milk powder into the cup
- User preference for adding sugar is
 - Yes: Add sugar into the cup
 - No: Do not add sugar into the cup
- Add boiled water into the cup
- Stir 1 minute time
- Add tea bag into the cup and keep 1 minute time
- Remove the tea bag from the cup
- Stir 1 minute time to mix tea with milk

Pay your attention for the sequential flow of tasks. You need to consider about the correct order of organizing the tasks.

Can you control the amount of sugar and milk powder in this case?



Let's Prepare a Cup of Milk Tea with More Control of Tasks

- What output do you need to generate? A cup of tea
- What input do you have/ need? Sugar, Milk, Tea, Water , Cup
- What form will it be in? Dry sugar, Tea bag, Milk powder, Boiled water , Empty cup
- Will the program have a list of tasks that needs to be done repeatedly (LOOP)? Yes
- If it has repeated tasks;
 - what will be done within it: Adding sugar and milk powder
 - What controls the loop? Number of teaspoons
 - What needs to be done before the loop? Have to have boiled water
 - What needs to be done after the loop? Continue with other major tasks.
- What are the other major tasks of the program?
 - Take the empty cup
 - Add milk powder into the cup
 - User preference for adding sugar is
 - Yes: Add sugar into the cup
 - No: Do not add sugar into the cup
 - Add boiled water into the cup
 - Stir 1 minute time
 - Add tea bag into the cup and keep 1 minute time
 - Remove the tea bag from the cup
 - Stir 1 minute time to mix tea with milk

Let's Prepare a Cup of Milk Tea with More Control of Tasks (Cont...)

- You have to add loops if you have repetitive tasks in your problem to be solved; e.g. adding sugar and adding milk powder
- Loop constructs of five building blocks
 - Loop control variable
 - Initial value of the loop control variable
 - Loop control condition
 - Repetitive statements
 - Increment the loop control variable

Let's Prepare a Cup of Milk Tea with More Control of Tasks (Cont...)

- Add milk powder in to the cup
 - Loop control variable → Tea spoon count
 - Initial value of the loop control variable → 0
 - Loop control condition → tea spoon count == 2
 - Repetitive statements → Add a tea spoon of milk
 - Increment the loop control variable → incrementing the tea spoon count

Let's Prepare a Cup of Milk Tea with More Control of Tasks (Cont...)

- Add milk powder in to the cup
 - tea spoon count for milk powder is 0

Tea spoon
count == 2



- Add a tea spoon of milk
- Increment the tea spoon count for milk by 1

Let's Prepare a Cup of Milk Tea with More Control of Tasks (Cont...)

- Add sugar in to the cup
 - Loop control variable → Tea spoon count
 - Initial value of the loop control variable → 0
 - Loop control condition → tea spoon count == 3
 - Repetitive statements → Add a tea spoon of sugar
 - Increment the loop control variable → incrementing the tea spoon count

Let's Prepare a Cup of Milk Tea with More Control of Tasks (Cont...)

- Add sugar in to the cup
 - tea spoon count for sugar is 0

Tea spoon
count == 3



- Add a tea spoon of sugar
- Increment the tea spoon count for sugar by 1

Let's Prepare a Cup of Milk Tea with More Control of Tasks (Cont...)

- Loops

- Add milk powder in to the cup

- tea spoon count for milk powder is 0

- Add a tea spoon of milk

- Increment the tea spoon count for milk by 1

- Add sugar in to the cup

- tea spoon count for sugar is 0

- Add a tea spoon of sugar

- Increment the tea spoon count for sugar by 1

Tea spoon
count ==2



Tea spoon
count ==3



Full Recipe of Preparing a Cup of Milk Tea with Sugar Preference

- Take the empty cup
 - tea spoon count for milk powder is 0
 - Add a tea spoon of milk
 - Increment the tea spoon count for milk by 1
- User preference for adding sugar is

Tea spoon
count == 2



- Yes:

- tea spoon count for sugar is 0
- Add a tea spoon of sugar
- Increment the tea spoon count for sugar by 1

Tea spoon
count == 3



- No: Do not add sugar in to the cup

- Add boiled water until the cup filled
- Stir 1 minute time
- Add tea bag in to the cup and keep 1 minute time
- Remove the tea bag from the cup
- Stir 1 minute time to mix tea with milk



The Structure Theorem

- The structure theorem forms the basic framework for structured programming.
- It states that it is possible to write any computer program by using only three basic control structures such as:
 - Sequence
 - Selection and
 - Repetition / Iteration

Sequence

- The sequence control structure is the straight forward execution of one processing step after another.
- E.g.
 - Statement 1
 - Statement 2
 - Statement 3
- Sequence instructions will be executed in the order in which it appears.

Selection

- The selection control structure is the presentation of a condition and the choice between two actions, the choice depending on whether the condition is true or false.
- This construct represents the decision making abilities of the computer.
- After processing selection part control is passed to the next processing step.

Repetition

- The repetition control structure can be defined as the presentation of a set of instructions to be performed repeatedly, as long as a condition is true.
- After processing repetition part successfully the control is passed to the next processing.
- You need to accurately write and test the accuracy of the repetition structure because your mistake can causes get endless repetition structures.

Activity

- Write the task list for a vending machine that can make either a cup of milk tea or a cup of coffee one at a time



Objectives - Revisit

- Define the phases of the program development life cycle.
- Prepare task lists for solving given problems.
- Define structured program theorem.
- Apply structured program theorem when preparing a task list for a given problem.

References

- Chapter 01 of Computer Program Design, Elizabeth A. Dickson.

Q & A

Next: Flow Charts and Design Structures